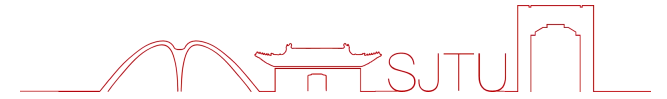




上海交通大学
SHANGHAI JIAO TONG UNIVERSITY



FaaS Lab



饮水思源 · 爱国荣校



1

Introduction and background

2

Hands on Docker

3

Hands on K8S

4

Hands on FaaS

01

Introduction and background

- What is Serverless?
- What is FaaS (Function-as-a-Service)?
- Why use FaaS?
- FaaS vs. Traditional Models



Introduction and background

Why use FaaS ?

- **Core pain points:** Dependency Hell and environment inconsistency.

Developer A



Developer A: "My code depends on Python 3.7 and OpenSSL 1.0, and it works fine."

Server Operations B



Server Operations B: "No! The production server system comes with Python 2.7, and for security reasons, OpenSSL must be version 1.1! Your code won't run!"

Developer C



Developer C: "I need to use a library called 'lib-X', but it has a version conflict with another dependency library of Developer A!"



Introduction and background

- ④ So how do you solve dependency hell and version inconsistencies?
 - **Real-world solution:** Standardize!
 - **Core idea:** Use a standardized box to seal everything, no matter what's inside. The transport only deals with the box, not the contents.



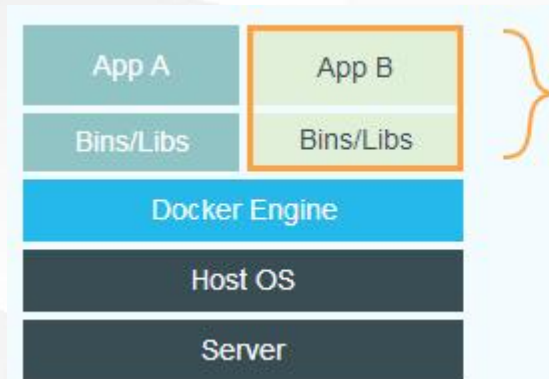
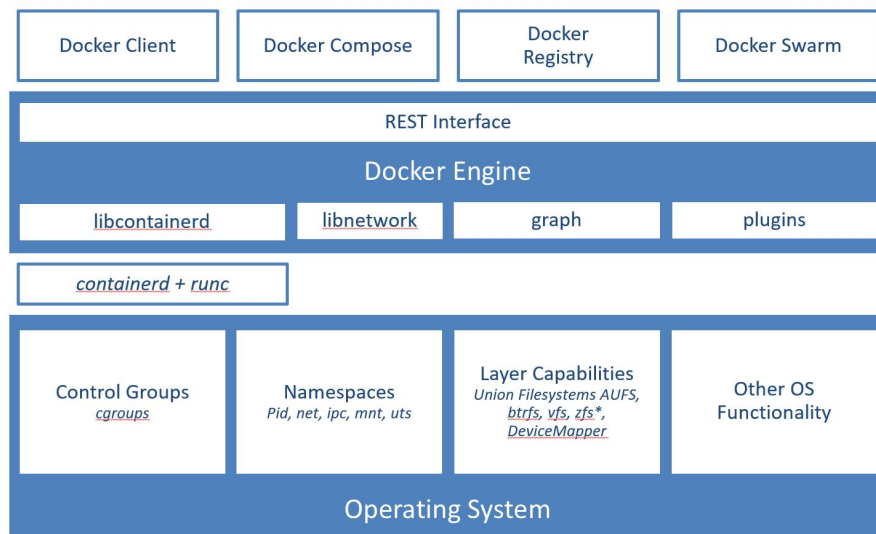
Introduction and background



Docker - Application packaging expert

- What is Docker:
 - **Definition:** Docker is an open source application container engine.
 - **What it does:** Packages your application along with all its dependencies (libraries, configuration files, runtime, etc.) into an isolated, portable container.
 - **Result:** Build once, run anywhere.

Architecture In Linux



Docker

The Docker Engine container comprises just the application and its dependencies. It runs as an isolated process in userspace on the host operating system, sharing the kernel with other containers. Thus, it enjoys the resource isolation and allocation benefits of VMs but is much more portable and efficient.





Introduction and background

New Problems after Docker

- An application consists of multiple containers (Web, DB, Cache). How do they discover and communicate with each other?
- How to achieve high availability of services? If a container fails, how does the system "self-heal"?
- How to automatically expand or shrink the number of containers according to traffic?
- How to update the online service version safely and smoothly?



Introduction and background

Kubernetes - The "conductor" of containers

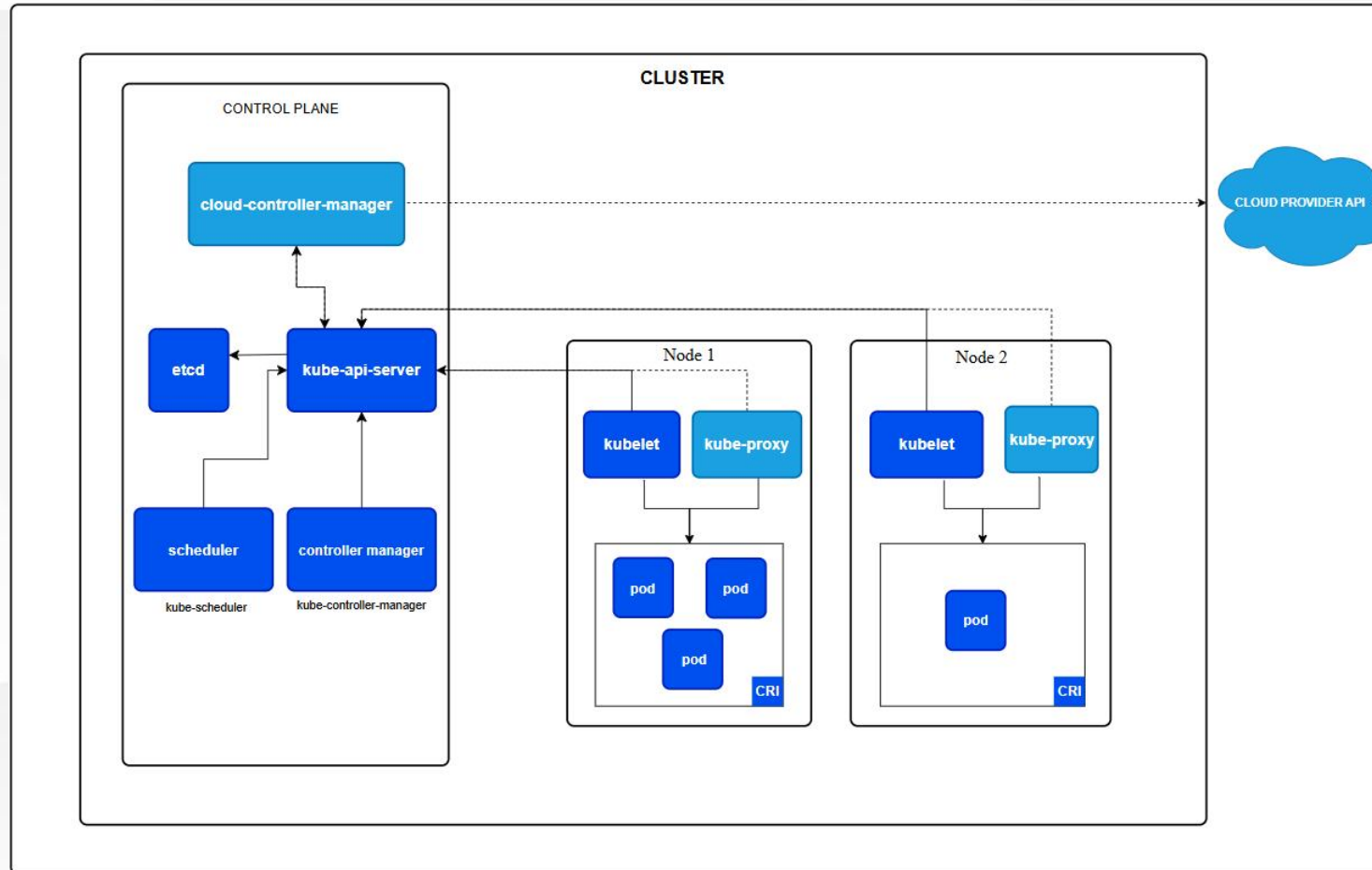
- Definition: Kubernetes (K8s) is an open source system for automating the deployment, expansion, and management of containerized applications.
- Core idea: Declarative API. You tell K8s the desired final state (Desired State), and K8s will continue to work to make the actual state of the system (Actual State) match it.



Introduction and background

K8s core architecture and concepts

- Macro structure:





Introduction and background

K8s core architecture and concepts

- Core concepts:
 - **Pod**: The smallest deployment unit in K8s. Think of it as a "pod" that can contain one or more closely related containers.
 - **Deployment**: "Deployment declaration", defines the desired state of the application, such as "I need 3 Nginx Pod replicas". It ensures the number of Pods by managing ReplicaSet.
 - **Service**: "Service discovery and load balancer". It provides a fixed, unified access portal (a virtual IP) for a group of Pods with the same function.



Introduction and background

🕒 New “troubles” and the emergence of FaaS

- The reality is that although K8s is powerful, the learning curve is still steep for ordinary developers. If I just want to implement a simple function, do I really need to care about the number of Pod replicas, update strategies and other things? I don't want to care! I just want to write my core business logic code. FaaS, Function as a Service, was born to solve this "last worry". It is built on K8s and aims to completely free developers from the complexity of infrastructure.



FaaS vs. Traditional Models

Feature	Traditional Backend	Microservices Architecture	FaaS (Function-as-a-Service)
Deployment Model	Long-running services	Each service deployed separately	Each function invoked on demand
Resource Utilization	Consumes persistent resources	Fine-grained services but still resident	Run on demand, released after execution
Management Complexity	Medium to high	Requires container orchestration	Very low, handled by platform
Startup Time	Fast	Fast	Slower (Cold Start issue)
Typical Use Cases	Web applications	Gateways, modular system components	Async tasks, event handling, lightweight APIs



02

Hands on Docker



- ☉ The core value of Docker: Pack the application + all dependencies into a box to achieve "build once, run everywhere".
- ☉ Introduction to key concepts:
 - **Dockerfile**: A "blueprint" or "instruction manual" that defines how to build the application environment step by step.
 - **Image**: A read-only "static template" that contains the application and all its dependencies.
 - **Container**: A running instance of an image, an isolated, executable "sandbox".
 - **Port Mapping**: A "bridge" that connects the host's port to the container's port, allowing the outside world to access the services in the container.
 - **Docker Network**: A "virtual private network" that allows containers to communicate with each other by name.



Hands-on practice: containerized front-end and back-end separation applications

- Step 1: Prepare the "backend API" service
 - Create a simple API named api.py that returns a JSON data.

```
api.py x
11 #api.py
10 from http.server import BaseHTTPRequestHandler, HTTPServer
9 import json
8
7 class ApiServer(BaseHTTPRequestHandler):
6     def do_GET(self) -> None:
5         self.send_response(code=200)
4         self.send_header(keyword="Content-type", value="application/json")
3         self.end_headers()
2         response: dict[str, str] = {
1             "message": "Hello from the API!", "version": "1.0"
12         }
1         self.wfile.write(buffer: bytes(o: json.dumps(obj: response), "utf-8"))
2
3 if __name__ == "__main__":
4     webServer: HTTPServer = HTTPServer(
5         server_address: ("0.0.0.0", 5000),
6         RequestHandlerClass: ApiServer
7     )
8     print(*values: "API server started on port 5000")
9     webServer.serve_forever()
```



Hands on Docker

🕒 Hands-on practice: containerized front-end and back-end separation applications

- Step 1: Prepare the "backend API" service
 - Write api.Dockerfile for the API service.

```
api.Dockerfile
5 # api.Dockerfile
4 FROM python:3.9-slim
3 WORKDIR /app
2 COPY api.py .
1 EXPOSE 5000
6 CMD [ "python", "api.py" ]
```



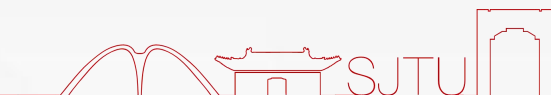


Hands-on practice: containerized front-end and back-end separation applications

- Step 2: Prepare the "front-end web" application
 - Create app.py and let it call the backend API. This requires the requests library.

```
app.py
24 # app.py
23 from http.server import BaseHTTPRequestHandler, HTTPServer
22 import os
21 import requests # 新增依赖
20 # API服务的地址, 通过环境变量获取
19 API_URL = os.environ.get(key: 'API_URL', 'http://localhost:5000')
18 class MyServer(BaseHTTPRequestHandler):
17     def do_GET(self) -> None:
16         self.send_response(code: 200)
15         self.send_header(keyword: "Content-type", value: "text/html")
14         self.end_headers()
13
12         api_data = "Could not connect to API"
11         try:
10             # 调用API服务
9             response: Response = requests.get(url: API_URL)
8             if response.status_code == 200:
7                 api_data = response.json().get('message', 'No message from API')
6             except requests.exceptions.ConnectionError as e:
5                 api_data: str = f"Connection Error: {e}"
4
3             pod_name: str = os.environ.get(key: 'HOSTNAME', 'localhost')
2             self.wfile.write(
1             buffer: bytes(
25             o: f"<html><body><h1>WebApp on {pod_name} says:</h1><h2>{api_data}</h2></body></html>",
1             "utf-8"
2             )
3             )
4
5 if __name__ == "__main__":
6     webServer = HTTPServer(
7     ("0.0.0.0", 8080),
8     MyServer
```

```
5 if __name__ == "__main__":
6     webServer = HTTPServer(
7     ("0.0.0.0", 8080),
8     MyServer
9     )
10    print("WebApp server started on port 8080")
11    webServer.serve_forever()
```





Hands on Docker

- Hands-on practice: containerized front-end and back-end separation applications
 - Step 2: Prepare the "front-end web" application
 - Prepare the front-end Dockerfile and add the steps to install the requests library

```
Dockerfile 1x
8 # Dockerfile
7 FROM python:3.9-slim
6 WORKDIR /app
5 # 新增：安装requests库
4 RUN pip install requests
3 COPY app.py .
2 EXPOSE 8080
1 CMD [ "python", "app.py" ]
9
```





🕒 Hands-on practice: containerized front-end and back-end separation applications

- Step 3: Build the image and create a network
 - Build two images:

```
docker build -t my-api-app:v1 -f api.Dockerfile .  
docker build -t my-web-app:v1 .
```

- Create a dedicated Docker network:

```
docker network create my-app-net
```



Hands on Docker

🕒 Hands-on practice: containerized front-end and back-end separation applications

- Step 4: Run the container and verify communication
 - Run the backend API container first:

```
# --network 指定网络, --name 给容器一个固定的名字my-api  
docker run -d --network my-app-net --name my-api my-api-app:v1
```

- Then run the front-end web container:

```
# 同样加入网络, 并通过环境变量告诉它API的地址就是 http://my-api:5000  
# -p 8080:8080 展示端口暴露: 将主机的8080端口映射到容器的8080端口  
docker run -p 8080:8080 --network my-app-net -e API_URL="http://my-api:5000" --name web-instance my-web-app:v1
```





🕒 Hands-on practice: containerized front-end and back-end separation applications

- Step 4: Run the container and verify communication
 - Verification results:

```
[hzk@hzk-VMware-Virtual-Platform docker_project]$ curl -v http://localhost:8080
* Host localhost:8080 was resolved.
* IPv6: ::1
* IPv4: 127.0.0.1
*   Trying [::1]:8080...
* Connected to localhost (::1) port 8080
> GET / HTTP/1.1
> Host: localhost:8080
> User-Agent: curl/8.5.0
> Accept: */*
>
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< Server: BaseHTTP/0.6 Python/3.9.23
< Date: Sun, 06 Jul 2025 07:26:09 GMT
< Content-type: text/html
<
* Closing connection
<html><body><h1>WebApp on 252fcbfe1a81says:</h1><h2>'Hello from the API!</h2></body></html>
```





03

Hands on K8S



- 🕒 **New question:** How to communicate reliably when the container IP changes? How to ensure high availability of services? How to upgrade smoothly?
- 🕒 **The core value of K8s:** automated operation and maintenance, service discovery and powerful orchestration capabilities.
- 🕒 Introduction to key concepts:
 - **Deployment:** Ensures that the application always has a specified number of replicas running, and is responsible for application updates and self-healing.
 - **Service:** Provides a fixed, unified access portal and load balancing for a group of Pods with the same functions.
 - **K8s DNS:** Domain name service within the cluster, allowing services to communicate through names (such as my-api-service) instead of mutable IP addresses.



Hands-on practice: experience the magic of K8s orchestration

- Step 1: Prepare Kubernetes deployment files
 - Create a file named k8s-deployment.yaml, which will define the deployment rules for both the backend API and the frontend web application.

```
k8s-deployment...  
22 # k8s-deployment.yaml  
21 apiVersion: apps/v1  
20 kind: Deployment  
19 metadata:  
18   name: my-api-deployment  
17 spec:  
16   replicas: 1  
15   selector:  
14     matchLabels:  
13       app: my-api  
12   template:  
11     metadata:  
10       labels:  
9         app: my-api  
8     spec:  
7       containers:  
6         - name: api-server  
5           image: my-api-app:v1  
4           imagePullPolicy: IfNotPresent # 优先使用本地镜像  
3           ports:  
2             - containerPort: 5000  
1 ---  
23 apiVersion: apps/v1  
1 kind: Deployment  
2 metadata:  
3   name: my-web-app-deployment  
4 spec:  
5   replicas: 2 # Web应用初始运行2个副本  
6   selector:  
7     matchLabels:  
8       app: my-web-app
```

```
k8s-deployment...  
14     imagePullPolicy: IfNotPresent # 优先使用本地镜像  
13     ports:  
12       - containerPort: 5000  
11 ---  
10 apiVersion: apps/v1  
9 kind: Deployment  
8 metadata:  
7   name: my-web-app-deployment  
6 spec:  
5   replicas: 2 # Web应用初始运行2个副本  
4   selector:  
3     matchLabels:  
2       app: my-web-app  
1   template:  
33     metadata:  
1       labels:  
2         app: my-web-app  
3     spec:  
4       containers:  
5         - name: web-server  
6           image: my-web-app:v1  
7           imagePullPolicy: IfNotPresent # 优先使用本地镜像  
8           ports:  
9             - containerPort: 8080  
10          env:  
11            - # 关键: API地址直接使用K8s Service的名字进行通信  
12              name: API_URL  
13              value: "http://my-api-service:5000"
```



Hands-on practice: experience the magic of K8s orchestration

- Step 2: Prepare Kubernetes service files
 - Create a file named k8s-service.yaml, which creates services for the front-end and back-end applications respectively to achieve service discovery and external access.

```
k8s-service.yaml
24 # k8s-service.yaml
23 apiVersion: v1
22 kind: Service
21 metadata:
20 | .name: my-api-service # API服务, 仅对集群内部可见
19 spec:
18 | .type: ClusterIP # 默认类型, 提供一个稳定的内部IP
17 | .selector:
16 | | .app: my-api
15 | .ports:
14 | | .port: 5000
13 | | .targetPort: 5000
12 ---
11 apiVersion: v1
10 kind: Service
9 metadata:
8 | .name: my-web-app-service # Web应用服务, 需要对外暴露
7 spec:
6 | .type: NodePort # 暴露服务到节点的一个静态端口, 方便外部访问
5 | .selector:
4 | | .app: my-web-app
3 | .ports:
2 | | .port: 8080
1 | | .targetPort: 8080
25 | | .nodePort: 30080 # 指定一个固定的端口
```





Hands on K8S

Hands-on practice: experience the magic of K8s orchestration

- Step 3: Deploy to Kubernetes and verify service discovery
 - Application deployment and services:

```
kubectl apply -f k8s-deployment.yaml  
kubectl apply -f k8s-service.yaml
```



Hands on K8S

Hands-on practice: experience the magic of K8s orchestration

- Step 3: Deploy to Kubernetes and verify service discovery
 - **Check the status:** `kubectl get pods -o wide`

```
[hzk@hzk-VMware-Virtual-Platform k8s_lab]$ kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED	NODE	READINESS	GATES
my-api-deployment-cb77dcdbf-krxtx	1/1	Running	0	2m28s	10.244.97.70	hzk-vmware-virtual-platform	<none>		<none>	
my-web-app-deployment-84b9c9cc5d-76msg	1/1	Running	0	2m28s	10.244.97.68	hzk-vmware-virtual-platform	<none>		<none>	
my-web-app-deployment-84b9c9cc5d-fmf92	1/1	Running	0	2m28s	10.244.97.69	hzk-vmware-virtual-platform	<none>		<none>	

- **Verify communication:** Open a browser and visit `http://<your K8s node IP>:30080`.

```
[hzk@hzk-VMware-Virtual-Platform k8s_lab]$ curl -v http://192.168.58.130:30080
* Trying 192.168.58.130:30080...
* Connected to 192.168.58.130 (192.168.58.130) port 30080
> GET / HTTP/1.1
> Host: 192.168.58.130:30080
> User-Agent: curl/8.5.0
> Accept: */*
>
* HTTP 1.0, assume close after body
< HTTP/1.0 200 OK
< Server: BaseHTTP/0.6 Python/3.9.23
< Date: Sun, 06 Jul 2025 08:16:47 GMT
< Content-type: text/html
<
* Closing connection
<html><body><h1>WebApp on my-web-app-deployment-84b9c9cc5d-fmf92says:</h1><h2>'Hello from the API!</h2></body></html>[
```





Hands-on practice: experience the magic of K8s orchestration

- Step 4: Verify Self-Healing
 - Open a new terminal and monitor the status of the Pod in real time:

```
[hzk@hzk-VMware-Virtual-Platform ~]$ kubectl get pods -w
```

NAME	READY	STATUS	RESTARTS	AGE
my-api-deployment-cb77dcbdf-krxtx	1/1	Running	0	10m
my-web-app-deployment-84b9c9cc5d-76msg	1/1	Running	0	10m
my-web-app-deployment-84b9c9cc5d-fmf92	1/1	Running	0	10m

- In the original terminal, manually delete a running Web App Pod:

```
[hzk@hzk-VMware-Virtual-Platform k8s_lab]$ kubectl delete pod my-web-app-deployment-cb77dcbdf-krxtx
```

Error from server (NotFound): pods "my-web-app-deployment-cb77dcbdf-krxtx" not found

```
[hzk@hzk-VMware-Virtual-Platform k8s_lab]$ kubectl delete pod my-web-app-deployment-84b9c9cc5d-76msg
```

pod "my-web-app-deployment-84b9c9cc5d-76msg" deleted

```
[hzk@hzk-VMware-Virtual-Platform ~]$ kubectl get pods -w
```

NAME	READY	STATUS	RESTARTS	AGE
my-api-deployment-cb77dcbdf-krxtx	1/1	Running	0	10m
my-web-app-deployment-84b9c9cc5d-76msg	1/1	Running	0	10m
my-web-app-deployment-84b9c9cc5d-fmf92	1/1	Running	0	10m
my-web-app-deployment-84b9c9cc5d-76msg	1/1	Terminating	0	12m
my-web-app-deployment-84b9c9cc5d-vk8p4	0/1	Pending	0	0s
my-web-app-deployment-84b9c9cc5d-vk8p4	0/1	Pending	0	0s
my-web-app-deployment-84b9c9cc5d-vk8p4	0/1	ContainerCreating	0	0s
my-web-app-deployment-84b9c9cc5d-vk8p4	0/1	ContainerCreating	0	0s
my-web-app-deployment-84b9c9cc5d-vk8p4	1/1	Running	0	2s





Hands on K8S

Hands-on practice: experience the magic of K8s orchestration

- Step 5: Verify Rolling Update
 - Prepare the V2 version of the application:
 - Modify app.py and slightly change the returned HTML content, such as <h1>WebApp v2 on
 - Rebuild the Docker image of the web application and add a new tag v2:

```
docker build -t my-web-app:v2 .
```

- Update the Deployment file:
 - Modify the image tag in the my-web-app-deployment section of k8s-deployment.yaml:

```
containers:
- name: web-server
  image: my-web-app:v2
  imagePullPolicy: IfNotPresent # 优先使用本地镜像
  ports:
  - containerPort: 8080
  env:
```





Hands-on practice: experience the magic of K8s orchestration

- Step 5: Verify Rolling Update
 - Prepare the V2 version of the application:
 - Modify app.py and slightly change the returned HTML content, such as <h1>WebApp v2 on
 - Rebuild the Docker image of the web application and add a new tag v2:

```
docker build -t my-web-app:v2 .
```

- Update the Deployment file:
 - Modify the image tag in the my-web-app-deployment section of k8s-deployment.yaml:

```
containers:
- name: web-server
  image: my-web-app:v2
  imagePullPolicy: IfNotPresent # 优先使用本地镜像
  ports:
  - containerPort: 8080
  env:
```





Hands on K8S

Hands-on practice: experience the magic of K8s orchestration

- Step 5: Verify Rolling Update
 - Apply the update and observe the process:
 - In one terminal, use watch and curl to continuously access the service and observe the changes in the returned content:

```
watch -n 1 curl http://<你的K8s节点IP>:30080
```

```
Every 1.0s: curl http://192.168.58.1... hzk-VMware-Virtual-Platform: Sun Jul  6 16:34:40 2025

  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
  0     0    0     0    0     0     0     0  --:--:-- --:--:-- --:--:--    0 100  118    0
 118     0    0 29979    0 --:--:-- --:--:-- --:--:-- 39333
<html><body><h1>WebApp on my-web-app-deployment-84b9c9cc5d-vk8p4says:</h1><h2>'Hello from the
API!'</h2></body></html>
```





Hands-on practice: experience the magic of K8s orchestration

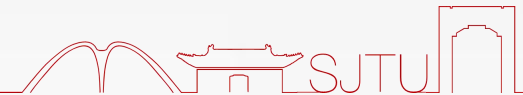
- Step 5: Verify Rolling Update
 - Apply the update and observe the process:
 - In another terminal, apply the updated YAML file:

```
[hzk@hzk-VMware-Virtual-Platform k8s_lab]$ kubectl apply -f k8s-deployment.yaml  
  
deployment.apps/my-api-deployment unchanged  
deployment.apps/my-web-app-deployment configured
```

- In the third terminal, observe the status of the rolling update:

```
[hzk@hzk-VMware-Virtual-Platform ~]$ kubectl rollout status deployment/my-web-app-deployment  
deployment "my-web-app-deployment" successfully rolled out
```

```
{Every 1.0s: curl http://192.168.58.130:30080 hzk-VMwar  
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current  
           Dload  Upload   Total     Spent    Left     Speed  
0     0     0     0     0     0     0     0  --:--:-- --:--:-- --:--:--  
0     0     0     0     0     0     0     0  --:--:-- --:--:-- --:--:-- 30250  
<html><body><h1>WebApp V2 on my-web-app-deployment-775df9c7ff-qm86ksays:</h1><
```





Hands on K8S

Hands-on practice: experience the magic of K8s orchestration

- Step 6: Verify elastic scaling
 - Manual expansion:

```
[hzk@hzk-VMware-Virtual-Platform k8s_lab]$ kubectl scale deployment my-web-app-deployment --replicas=5  
deployment.apps/my-web-app-deployment scaled
```

- View Status:

```
[hzk@hzk-VMware-Virtual-Platform k8s_lab]$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
my-api-deployment-cb77dcbdf-krxtx	1/1	Running	0	39m
my-web-app-deployment-775df9c7ff-6j46d	1/1	Running	0	26s
my-web-app-deployment-775df9c7ff-74vh7	1/1	Running	0	26s
my-web-app-deployment-775df9c7ff-8cqvj	1/1	Running	0	26s
my-web-app-deployment-775df9c7ff-hbws6	1/1	Running	0	3m10s
my-web-app-deployment-775df9c7ff-qm86k	1/1	Running	0	3m12s

```
[hzk@hzk-VMware-Virtual-Platform k8s_lab]$
```



04

Hands on FaaS

- Getting Started with Function Deployment and Calling
- Function dependency management with Python examples
- Communicating with external systems
- Concurrent calls and performance tuning
- Function Composition and Orchestration Practice



Introduction to basic usage of wsk CLI tool

- What is wsk CLI?
 - Official command line tool provided by OpenWhisk
 - Used to manage actions, triggers, rules, packages, etc.
 - Communicate with the platform through REST API

Operation	Command Example
Create an action	<code>wsk action create <name> <code></code>
Update an action	<code>wsk action update <name> <code></code>
Delete an action	<code>wsk action delete <name></code>
Invoke an action	<code>wsk action invoke <name> [--param <key> <value>]</code>
List all actions	<code>wsk action list</code>
View execution logs	<code>wsk activation list + wsk activation get <activationId></code>



Hello Function



Hands on FaaS

🕒 Writing your first function

- Let's take the simplest Python function as an example (hello.py):

```
python

# hello.py

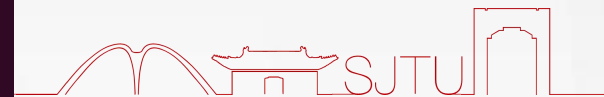
def main(params):
    name = params.get("name", "World")
    return { "greeting": f"Hello, {name}!" }
```

- Creating an action using the CLI:

```
[hzk@hzk-VMware-Virtual-Platform hands-on-faas]$ ls
hello.py
[hzk@hzk-VMware-Virtual-Platform hands-on-faas]$ cat hello.py
# hello.py
def main(params):
    name = params.get("name", "World")
    return { "greeting": f"Hello, {name}!" }

[hzk@hzk-VMware-Virtual-Platform hands-on-faas]$ wsk -i action create hello-python hello.py

ok: created action hello-python
```





Writing your first function

- Calling a function and passing parameters

```
[hzk@hzk-VMware-Virtual-Platform hands-on-faaS]$ wsk -i action invoke hello-python --param name 小明 --blocking --result
{
  "greeting": "Hello, 小明!"
}
```



Function dependency management with Python examples



Background: Stateless runtime of FaaS:

- The OpenWhisk function runs in a temporary container environment. The container may be destroyed after each execution. The running environment:
 - Does not contain any non-built-in third-party libraries
 - Each execution must "bring all dependencies"
 - So the dependencies need to be packaged together with the code



Hands on FaaS

🕒 Method 1: Use zip file to package Python functions and dependencies

- Example project structure:

```
[hzk@hzk-VMware-Virtual-Platform hands-on-faas]$ tree weather_action -L 1
weather_action
├── __main__.py
├── venv
└── weather_action.zip

2 directories, 2 files
```



Method 1: Use zip file to package Python functions and dependencies

- Pack the zip file and upload it

```
[hzk@hzk-VMware-Virtual-Platform weather_action]$ ls
venv  __main__.py  weather_action.zip
[hzk@hzk-VMware-Virtual-Platform weather_action]$ wsk -i action create weather weather_action.zip --kind python:3
ok: created action weather
```

- Calling a function

```
[hzk@hzk-VMware-Virtual-Platform weather_action]$ ls
venv  __main__.py  weather_action.zip
[hzk@hzk-VMware-Virtual-Platform weather_action]$ cat __main__.py
import requests
def main(args):
    city = args.get("city", "beijing")
    # 使用一个公开的天气API
    url = f"https://wttr.in/{city}?format=j1"
    try:
        resp = requests.get(url).json()
        condition = resp['current_condition'][0]
        return {"weather": f"{condition['weatherDesc'][0]['value']}", "temp_C": condition['temp_C']}
    except Exception as e:
        return {"error": str(e)}

[hzk@hzk-VMware-Virtual-Platform weather_action]$ wsk -i action invoke weather --result -p city shanghai
{
  "temp_C": "35",
  "weather": "Sunny"
}
[hzk@hzk-VMware-Virtual-Platform weather_action]$
```





Method 2: Customize Docker runtime

- When dependencies are complex or system-level libraries are required, the ZIP package will become bloated and powerless.
- Ultimate solution: dockerAction.
- Dockerfile example (domestic source optimization):

```
Dockerfile 1 x
1 FROM openwhisk/python3action
2 RUN pip install -i https://pypi.tuna.tsinghua.edu.cn/simple numpy pandas
3
```



Hands on FaaS

Method 2: Customize Docker runtime

- Deployment Commands

```
[hzk@hzk-VMware-Virtual-Platform hands-on-faas]$ wsk -i action create numpy-test numpy_action/numpy_action.py --docker bubble5945/numpy-runtime:v1
ok: created action numpy-test
```

- Calling the Function

```
[hzk@hzk-VMware-Virtual-Platform hands-on-faas]$ wsk -i action invoke numpy-test --result
{
  "input_array": [
    1,
    2,
    3,
    4,
    5
  ],
  "mean": 3.0,
  "sum": 15
}
```





Communicating with external systems



Accessing External APIs

- **Recap:** The weather function just demonstrated how to call an external HTTP API.
- **Key point:** The execution environment of a FaaS function is usually accessible to the public Internet.
- **Security issue:** How to manage API keys or other sensitive information securely?
- **Error example:** Hard-coded in the code.

```
# 错误示范!
```

```
API_KEY = "sk-xxxxxxxxxxxxxxxxxx"
```



Security Policy

- **Best Practice:** Use parameter passing, especially default parameters.
- Create a Package with default parameters:
 - We will create a package named `secure_pkg` and use the `-p` parameter to bind the API Key as the default parameter of the package.

```
[hzk@hzk-VMware-Virtual-Platform hands-on-faaS]$ wsk -i package create secure_pkg -p apiKey "faas"

ok: created package secure_pkg
```

- Create a function to read parameters:

```
my_api_code.py x
1 # my_api_code.py
2 def main(args) -> dict[str, str]:
3     ...# 从传入的参数字典中获取apiKey
4     ...# 如果调用时没有单独提供，它会从package的默认参数中自动继承
5     ...api_key: Unknown = args.get("apiKey")
6
7     ...if not api_key:
8         ...return {"error": "API Key not found."}
9
10    ...# 模拟使用API Key
11    ...print(*values: f"成功获取API Key, Key的最后3位是: ...{api_key[*values: -3:]}")
12    ...return {"status": "ok, key received"}
```





Security Policy

- **Create an Action in a Package:**

- Note that the Action name format is package name/Action name.

```
[hzk@hzk-VMware-Virtual-Platform hands-on-faas]$ wsk -i action create secure_pkg/myApiAction my_api_code.py
ok: created action secure_pkg/myApiAction
```

- **Call and verify**

```
[hzk@hzk-VMware-Virtual-Platform hands-on-faas]$ wsk -i action invoke secure_pkg/myApiAction --result --blocking
{
  "status": "ok, key received"
}
[hzk@hzk-VMware-Virtual-Platform hands-on-faas]$ wsk -i activation logs 7302a4c6ed37457a82a4c6ed37457a05
2025-07-04T13:52:53.557920014Z stdout: 成功获取API Key, Key的最后3位是: ...aas
```



Concurrent calls and performance tuning



🕒 Simulate concurrency and view performance

- Simulating concurrency (Python script load_test.py):

```
load_test.py x
load_test.py
1 import threading
2 import os
3 import time
4
5 ACTION_NAME = "weather" # 指定要测试的Action
6 CONCURRENCY = 10 # 并发数
7
8 def invoke_action(index) -> None:
9     ...print(*values: f"线程 {index}: 发起调用...")
10    ...os.system(command: f"wsk -i action invoke {ACTION_NAME} -p city=beijing")
11
12 threads: list[Unknown] = []
13 for i in range(stop: CONCURRENCY):
14     ...thread: Thread = threading.Thread(target=invoke_action, args=(i,))
15     ...threads.append(object: thread)
16     ...thread.start()
17     ...time.sleep(seconds: 0.1) # 错开一点点时间
18
19 for thread in threads:
20     ...thread.join()
21
22 print(*values: f"\n{CONCURRENCY} 个并发调用已全部发起！")
```





🕒 Simulate concurrency and view performance

- View performance:

```
[hzk@hzk-VMware-Virtual-Platform hands-on-faas]$ python3 load_test.py
线程 0: 发起调用...
线程 1: 发起调用...
ok: invoked /_/weather with id 7e77a2a92a674ad3b7a2a92a674ad3e3
ok: invoked /_/weather with id c62789aef89e4892a789aef89e689217
线程 2: 发起调用...
线程 3: 发起调用...
ok: invoked /_/weather with id 4eee36aab9ee4d48ae36aab9eebd4824
ok: invoked /_/weather with id 0b1a35cc5a6c45e49a35cc5a6cc5e44c
线程 4: 发起调用...
ok: invoked /_/weather with id e3cf74a28bdc477f8f74a28bdcf77fcb
线程 5: 发起调用...
ok: invoked /_/weather with id 228146ef4b4647ac8146ef4b4687ac5b
线程 6: 发起调用...
ok: invoked /_/weather with id 035bd125b1194ef09bd125b1194ef098
线程 7: 发起调用...
ok: invoked /_/weather with id 5541fbc901f442fb81fbc901f4f2fb13
线程 8: 发起调用...
ok: invoked /_/weather with id 398af4df77b945c78af4df77b985c737
线程 9: 发起调用...
ok: invoked /_/weather with id 5cd6dad2f5742cc96dad2f5792cc1a

10 个并发调用已全部发起!
```





Hands on FaaS

Simulate concurrency and view performance

- Analyze performance indicators:

First, list the most recent activation records and find the records generated by concurrent tests.

```
[hzk@hzk-VMware-Virtual-Platform hands-on-faaS]$ wsk -i activation get 398af4df77b945c78af4df77b985c737
```

```
ok: got activation 398af4df77b945c78af4df77b985c737
```

```
{
  "namespace": "guest",
  "name": "weather",
  "version": "0.0.1",
  "subject": "guest",
  "activationId": "398af4df77b945c78af4df77b985c737",
  "start": 1751638050477,
  "end": 1751638053241,
  "duration": 2764,
  "statusCode": 0,
  "response": {
    "status": "success",
    "statusCode": 0,
    "success": true,
    "result": {
      "temp_C": "30",
      "weather": "Clear"
    }
  },
  "logs": [],
  "annotations": [
    {

```

```
"annotations": [
  {
    "key": "path",
    "value": "guest/weather"
  },
  {
    "key": "waitTime",
    "value": 4135
  },
  {
    "key": "kind",
    "value": "python:3"
  },
  {
    "key": "timeout",
    "value": false
  },
  {
    "key": "limits",
    "value": {
      "concurrency": 1,
      "logs": 10,
      "memory": 256,
      "timeout": 60000
    }
  }
],
"publish": false
```





⌚ Performance Tuning and Cold Start

- **Tuning:**

- `$ wsk action update weather --memory 256 --timeout 60000`
- **Description:** Increasing memory (in MB) can usually improve CPU performance; increase the timeout (in milliseconds) to prevent complex tasks from timing out.

- **Cold Start Problem:**

- **Definition:** After a function has not been called for a long time, the platform needs to recreate the container when it is called for the first time, which takes a long time.
- **OpenWhisk mechanism:** OpenWhisk will keep the called container "warm" for a period of time to serve subsequent requests and avoid cold starts every time.
- **Observation:** If you call it multiple times in a row, you will find that `initTime` is only available for the first time, and this value will disappear or be 0 for subsequent calls.



Function Composition and Orchestration Practice



Sequence Action

- **Goal:** Create a workflow where the result of function A is used as the input of function B.
- Function A (process_data.py):

```
def main(args):  
    name = args.get("name", "guest")  
    print(f"Processing data for {name}")  
    # 此函数的输出, 将作为下一个函数的输入  
    return {"payload": f"Processed for {name}"}
```

- Function B (store_data.py)

```
def main(args):  
    # 接收上一个函数的输出  
    data = args.get("payload", "empty")  
    print(f"Storing data: '{data}' into database.")  
    return {"stored_data": data}
```



Sequence Action

- **Deploy these two functions separately**

```
[hzk@hzk-VMware-Virtual-Platform hands-on-faas]$ wsk -i action create process_data process_data.py
wsk -i action create store_data store_data.py
ok: created action process_data
ok: created action store_data
```

- **Create a Sequence:** Use the --sequence parameter to concatenate them. The order is important.

```
[hzk@hzk-VMware-Virtual-Platform hands-on-faas]$ wsk -i action create process_and_store --sequence process_data,store_data
ok: created action process_and_store
```



Sequence Action

- **Call Sequence and verify**

```
[hzk@hzk-VMware-Virtual-Platform hands-on-faas]$ wsk -i action invoke process_and_store --result -p name "Alice"
{
  "stored_data": "Processed for Alice"
}
```

- The output of process_data {"payload": "..."} is passed intact to store_data as the input parameter args.



Trigger + Rule

- **Goal:** Implement event-driven logic. When an event occurs, automatically trigger our workflow.
- **Create a trigger:**
 - `$ wsk -i trigger create new_user_event`
 - Create a rule to connect Trigger and Sequence
 - `$ wsk -i action create process_and_store --sequence process_data,store_data`
- **Manually trigger an event:**
 - `$ wsk -i trigger fire new_user_event --param name "Bob"`
- **Verify:**
 - `$ wsk -i activation list process_and_store -l 1`
 - `$ wsk -i activation result $ACTIVATION_ID`



Q&A

饮水思源 爱国荣校